

IR progression, translation engine, and
algorithms

Progressive IRs

Processing proceeds through a sequence of representations. Which stages are complete for a given repository is summarized in `../cogant/docs/reference/implementation_status.md` (partial areas include translation rules, state space, and Rust acceleration):

1. **Repo IR** — entities and relationships extracted from parsers.
2. **Program graph IR** — consolidated graph with deduplication and metadata.
3. **Semantic mapping IR** — output of the translation rule engine.
4. **State space IR** — variables, actions, transitions, observations.
5. **Process model IR** — higher-level control patterns where implemented.
6. **Validation IR** — coverage, confidence analysis, schema checks.

COGANT's program graph sits in the same conceptual space as

Translation rules

The **translation** stage applies declarative rules that refine roles, attach labels, and adjust confidence. Concurrency targets and layering are described in

`../cogant/docs/architecture/README.md`. The rule engine composes passes over the graph in a **fixpoint loop**: rules are re-applied until no new semantic mappings emerge or a configurable iteration cap is reached. The fixpoint loop follows the classical formulation of program analysis as fixpoint computation over a lattice of abstract states [cousot1977abstract]; in our setting the lattice is the set of semantic mappings partially ordered by inclusion, and the monotone operator is the composition of all registered rule applications. In the shipped implementation, each pass applies rules in descending `rule.priority`, and conflict resolution later compares (`rule_priority`, `confidence_score`) tuples when two mappings overlap, following the principle that edit representations should be composable [yin2019learning].

Fixpoint iteration, conflict resolution, and coverage

The shipped `TranslationEngine` in